(An Agile View of Process)
Chapter 4
Roger S. Pressman(6th Edition)

What is Agile?

Agile software development is a set of principles for software development in which requirements and solutions evolve through collaboration between self-organizing, cross-functional teams.

Agility can be defined as:

- Effective response to change
- Effective communication among all stakeholders
- Drawing the customer onto the team; eliminate the “us and them” attitude
- Organizing a team so that it is in control of the work performed
- Rapid, incremental delivery of software

The Manifesto for Agile Software Development

- Individuals and interactions: self-organization and motivation are important, as are interactions like colocation and pair programming.
- Working software: working software is more useful and welcome than just presenting documents to clients in meetings.

The Manifesto for Agile Software Development

- Customer collaboration: requirements cannot be fully collected at the beginning of the software development cycle, therefore continuous customer or stakeholder involvement is very important.
- Responding to change: agile methods are focused on quick responses to change and continuous development.

Principles of agile : (Hasan work day time to make food.)

The Agile Manifesto is based on twelve principles :

1. Highest priority -> satisfy the customer
2. Welcome changing requirements
3. Deliver working software frequently
4. Business people and developers must work together
5. Build projects around motivated individuals
6. Emphasize face-to-face conversation

Principles of agile : (People should accept societies best effort.)

7. Working software is the primary measure of progress
8. Agile processes promote sustainable development
9. Continuous attention to technical excellence and good design enhances agility
10. Simplicity – the art of maximizing the amount of work not done – is essential
11. The best designs emerge from self-organizing teams
12. The team tunes and adjusts its behavior to become more effective

Characteristics of Agile Software Development

-- Light Weighted methodology
-- Small to medium sized teams
-- vague and/or changing requirements
-- vague and/or changing techniques
-- Simple design
-- Minimal system into production

Agile Software Process – Three Key Assumptions

- Difficulty in predicting changes of requirements and customer priorities
- For many types of s/w, design and construction are interleaved
- Analysis, design, construction, and testing are not as predictable as we might like

Agile Software Process

- An agile process must be adaptable
- It must adapt incrementally
- Requires customer feedback
- An effective catalyst for customer feedback is an operational prototype

The Politics of Agile Development

- There is considerable debate about the benefits and applicability of agile software development
- No one is against agility. The real question is:
  - What is the best way to achieve it?

Human Factor

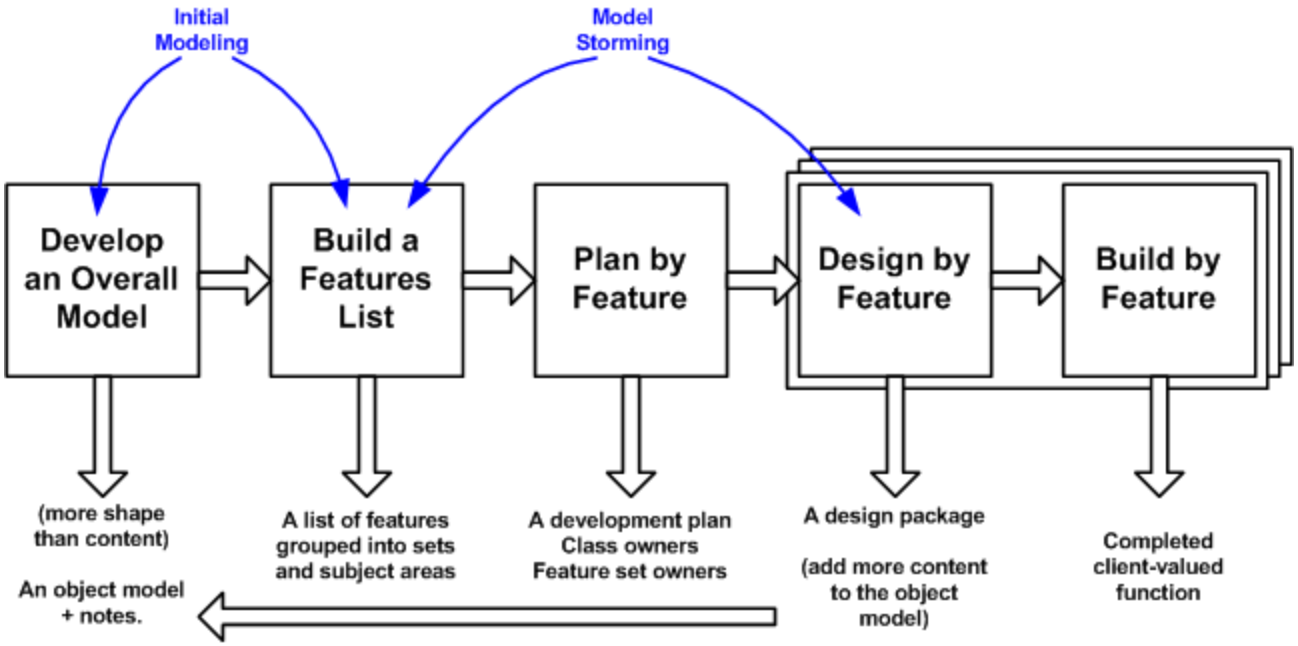
- Key point:
  - The Process molds to the needs of the people and team, not the other way around
- A number of key traits (characteristic) must exist among the people on an agile team and the team itself
  - Competence
  - Common focus
  - Collaboration
  - Decision-making ability
  - Fuzzy problem-solving ability
  - Mutual trust and respect
  - Self-organization

Agile Process Models

1. Extreme Programming (XP)
2. Adaptive Software Development (ASD)
3. Dynamic Systems Development Method (DSDM)
4. Scrum(**Scrum is a framework for project management that emphasizes teamwork, accountability and iterative progress toward a well-defined goal.**)
5. Crystal
6. Feature Driven Development (FDD)
7. Agile Modeling (AM)

Feature-Driven Development (**FDD**)

- Feature-Driven Development (**FDD**) is a client-centric, architecture-centric, and pragmatic **software** process.
- Features are an important aspect of **FDD**.

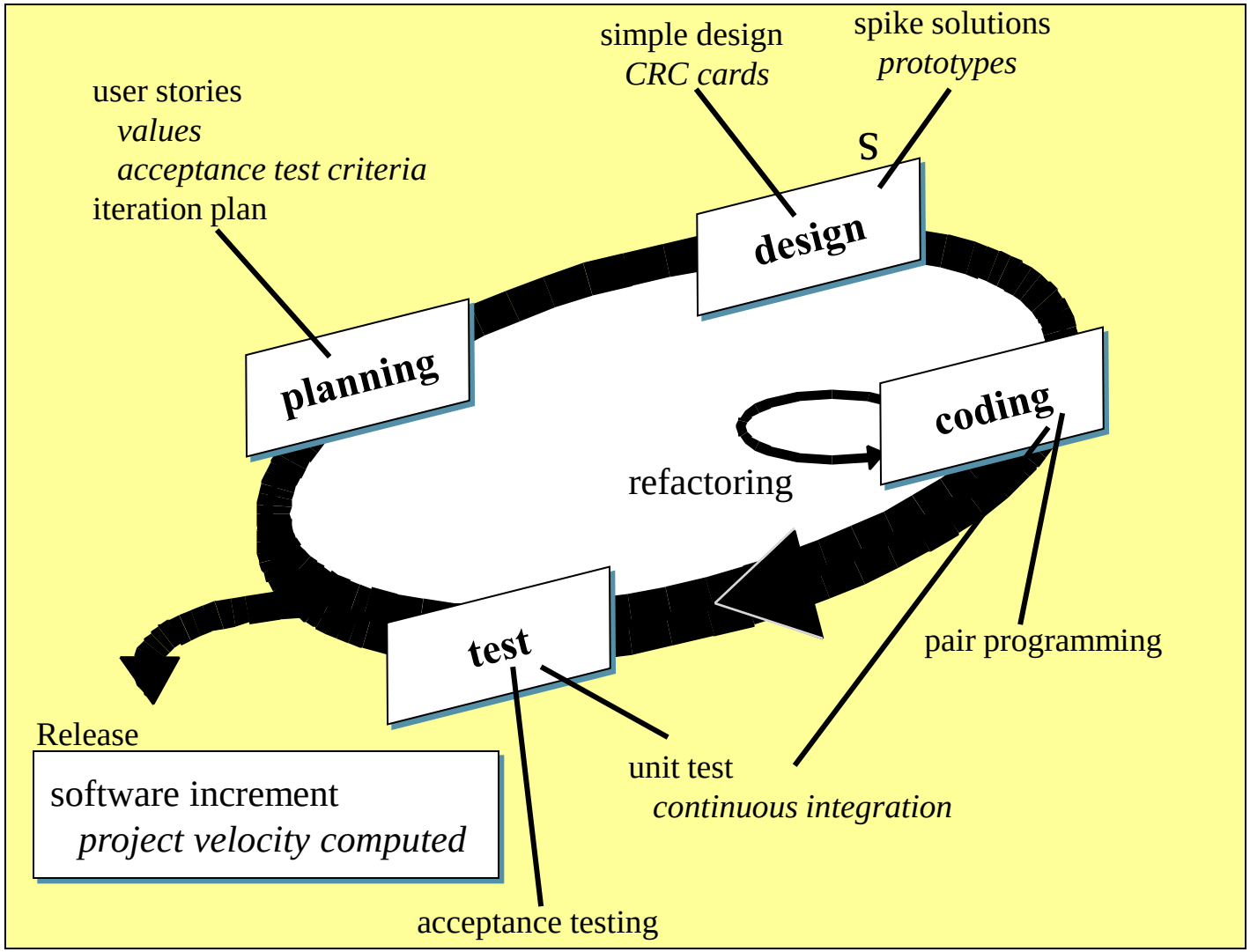


Copyright 2002-2005 Scott W. Ambler
Original Copyright S. R. Palmer & J.M. Felsing

Extreme Programming (XP) - 1

- The most widely used agile process, originally proposed by Kent Beck [BEC99]
- XP uses an object-oriented approach as its preferred development paradigm
- Defines four (4) framework activities
  - Planning
  - Design
  - Coding
  - Testing

Extreme Programming (XP) - 2



XP - Planning

- Begins with the creation of a set of stories (also called *user stories*)
- Each story is written by the customer and is placed on an index card
- The customer assigns a value (i.e. a priority) to the story
- Agile team assesses each story and assigns a cost
- Stories are grouped to for a deliverable increment
- A commitment is made on delivery date
- After the first increment "project velocity" is used to help define subsequent delivery dates for other increments

XP - Design

- Follows the KIS (keep it simple) principle
- Encourage the use of CRC (class-responsibility-collaborator) cards (**CRC cards** are a brainstorming tool used in the design of object-oriented software.)
- For difficult design problems, suggests the creation of "*spike solutions*"—a design prototype
- Encourages "*refactoring*"—an iterative refinement of the internal program design
- Design occurs both before and after coding commences

XP - Coding

- Recommends the construction of a series of unit tests for each of the stories before coding commences (begins)
- Encourages "*pair programming*"
  - Mechanism for real-time problem solving and real-time quality assurance
  - Keeps the developers focused on the problem at hand
- Needs continuous integration with other portions (stories) of the s/w, which provides a "smoke testing" environment (Chapter 13)

XP - Testing

- Unit tests should be implemented using a framework to make testing automated. This encourages a regression testing strategy.
- Integration and validation testing can occur on a daily basis
- *Acceptance tests*, also called *customer tests*, are specified by the customer and executed to assess (judge) customer visible functionality
- Acceptance tests are derived from user stories

Smoke Testing

- Smoke Testing, also known as “Build Verification Testing”,
- is a type of software testing that comprises of a non-exhaustive set of tests that aim at ensuring that the most important functions work. The results of this testing is used to decide if a build is stable enough to proceed with further testing.

Refs

- Introduction, 4.1 – 4.3.1
- http://www.extremeprogramming.org/map/project.html
- http://softwaretestingfundamentals.com/smoke-testing/